

Guided Local Search — an illustrative example in function optimisation

C Voudouris

The Guided Local Search method has been successfully applied to a number of hard combinatorial optimisation problems from the well-known TSP and QAP to real-world problems such as frequency assignment and workforce scheduling.

In this paper, it is demonstrated that the potential applications of GLS are not limited to optimisation problems of discrete nature but also to difficult continuous optimisation problems. Continuous optimisation problems arise in many engineering disciplines (such as electrical and mechanical engineering) in the context of analysis, design or simulation tasks. The problem examined gives an illustrative example of the behaviour of GLS, providing insights on the mechanisms of the algorithm.

1. Introduction

Guided Local Search, originally proposed by Voudouris and Tsang [1], is a general optimisation technique suitable for a wide range of combinatorial optimisation problems. Successful applications of the technique so far include practical problems such as frequency allocation [1], workforce scheduling [2], vehicle routing [3, 4], and also classic problems such as the travelling salesman problem (TSP) and the quadratic assignment problem (QAP) [5]. Guided Local Search (GLS) belongs to a class of techniques known as metaheuristics [6–8]. Prominent members of this class include tabu search [9–14], simulated annealing [15–18], GRASP [19], genetic algorithms [20–22], scatter search [10] and others. Metaheuristics aim at enhancing the performance of heuristic methods in solving large and difficult combinatorial optimisation problems.

In the case of GLS, the main focus is on the exploitation of problem and search-related information to effectively guide local search heuristics in the vast search spaces of NP-hard optimisation problems. This is achieved by augmenting the objective function of the problem to be minimised with a set of penalty terms which are dynamically manipulated during the search process to steer the heuristic to be guided. Higher goals, such as the distribution of the search effort to the areas of the search space according to the promise of these areas to contain high-quality solutions, can be expressed and pursued.

GLS is closely related to the frequency-based memory approaches introduced in tabu search [11, 23], extending

these approaches to take into account the quality of structural parts of the solution and also react to feedback from the local optimisation heuristic under guidance. This paper provides an illustrative example of how GLS works by explaining its use for solving an a non-convex optimisation problem.

2. Non-convex optimisation and global optimisation methods

Continuous optimisation problems arise in many engineering disciplines (such as electrical and mechanical engineering) in the context of analysis, design or simulation tasks. Particularly difficult problems are those with nonlinear multi-extremal cost functions (that is functions with many local minima). These problems, also known as non-convex optimisation problems [24], are difficult to solve using deterministic gradient-based algorithms used extensively elsewhere in continuous optimisation. Gradient algorithms can easily be trapped in the many local minima of the cost function, so failing to reach the global minimum.

Global optimisation (GO) methods which seek the global minimum are utilised to solve such problems. The most simple global optimisation algorithm is to run a gradient algorithm many times and from different starting points in the hope that the global minimum will be among the local minima obtained over the many runs. An example of such an algorithm is the variation of the sequential unconstrained minimisation technique suggested in Hiller

and Lieberman [24]. Many other GO algorithms exist which make use of gradient techniques or derive directly from general search methods such as genetic algorithms [25], simulated annealing [26], function smoothing [27], orthogonal arrays with the GRG algorithm [28] to name but a few.

3. Local Search for continuous optimisation problems

Recently and mainly driven by the use of genetic algorithms [20, 21, 25] in combinatorial optimisation, GO methods have been developed which deal with nonconvex optimisation as a combinatorial optimisation task. The idea is to convert the continuous problem to a discrete one by encoding the real variables of the cost function as binary strings. In the case of binary encoding, a binary string value is interpreted to represent an integer in base-2 notation. The mapping of the binary string to a real variable works as follows. The binary string value is first converted to the corresponding integer. This integer is then scaled by the appropriate coefficient to give a real value in the desired range (i.e. domain of variable) [20]. One binary string is used for each problem variable and combinatorial search is utilised to find these binary string configurations which, after decoding, result in the optimal value for the real-valued cost function. Increasing the number of bits used for representing each variable increases the accuracy of the solution but also results in an increase of the combinatorial search space.

Although binary encoding schemes were principally developed for genetic algorithms, they have also been used in the context of local search [29, 30]. To explain how local search operates in this case, let us consider the problem with two variables $x \in A \subset \mathbb{R}$ and $y \in B \subset \mathbb{R}$ and a function $f(x, y)$ to be minimised in $A \times B \subset \mathbb{R}^2$. A local search move flips the value of a bit in the binary string representing the solution (comprising the binary strings of the function's variables). In the x - y plane, bit flips translate to 'jumps' in either the x or y direction. The more significant the bit changed, the larger the step of the 'jump' performed. Local search starting from a random binary string examines all possible bit flips and performs that which results in the maximum reduction in cost (minimisation case). The new solution, if better, replaces the old solution and the procedure continues from there on until a solution is reached for which no further improvement is possible.

4. The sine envelope sine wave (F6) function

As mentioned in section 2, non-convex optimisation refers to nonlinear multi-extremal cost functions. An example of such a function, mentioned many times in the literature, is the sine envelope sine wave function also known as F6 [20, 29, 30]:

$$F6(x, y) = 0.5 + \frac{\sin^2 \sqrt{x^2 + y^2} - 0.5}{[1.0 + 0.001 \cdot (x^2 + y^2)]^2} \quad \dots (1)$$

minimised in the domain $[-100, 100] \times [-100, 100]$. F6 has been suggested as a benchmark for genetic algorithms [31].

A cross section of the function is shown in Fig 1. The global minimum of F6 is located at (0,0) where the function takes the value 0. The basin of the global minimum is very narrow and therefore difficult to reach unless a lucky start is made from within the domain of attraction of the global minimum. The many local minima of the function are arranged in concentric cycles around the global minimum forming an ideal trap for hill-climbing based techniques. In F6, local gradients provide limited (if any) information on the location of the global minimum. Guided Local Search may be exploited to help local search to escape from local minima and moreover distribute its search efforts in the search space.

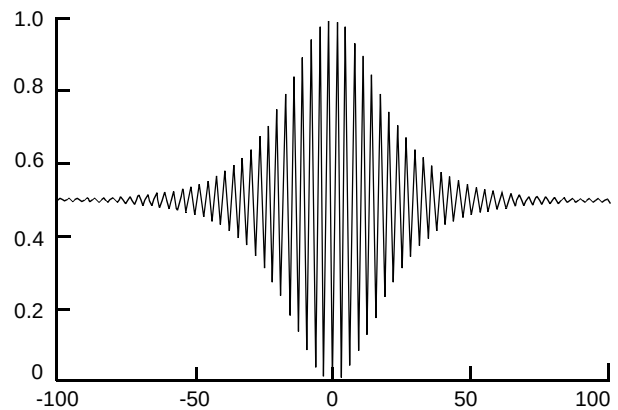


Fig 1 Cross section of F6 function.

5. Guided Local Search for global optimisation

GLS is iteratively posting constraints which modify the landscape and guide local search out of local minima and towards promising areas in the search space. Constraint posting in this problem could be based on information gathered during the search process. For example, if local search reaches a local minimum then an assumption can be made that the global minimum is unlikely to reside in the surrounding area. Constraints could be introduced that exclude this area from being searched in future iterations. These constraints are essentially soft because we cannot be sufficiently confident that local search thoroughly searches the space around a solution when this solution is visited.

A set of features is defined that allows solutions to be constrained. A feature can be any solution property represented by an indicator function [5]. A simple setting for global optimisation is to divide the domains of variables

into a number of non-overlapping and equally-sized intervals. Let us consider the variable $x \in (a, b)$. A set of features f_i , $i = 1, \dots, n$, can be defined by the intervals $(a_0 = a, a_1], (a_1, a_2], \dots, (a_{n-1}, a_n = b]$ as follows:

$$I_i(x) = \begin{cases} 1, & x \in (a_{i-1}, a_i) \\ 0, & \text{otherwise} \end{cases} \quad \dots (2)$$

To each feature f_i is attached a penalty parameter p_i to allow GLS to penalise solutions that are characterised by the feature such that they can be avoided. The cost function is augmented with penalty terms to form the augmented cost function. This function replaces the original function and it is minimised instead. The augmented version of F6 is defined as follows:

$$H(x, y) = F6(x, y) +$$

$$\lambda \left(\sum_{i=1}^n I_{xi}(x) \cdot p_{xi} + \sum_{j=1}^m I_{yj}(y) \cdot p_{yj} \right) \quad \dots (3)$$

where n is the number of features defined over the domain of x , m is the number of features defined over the domain of y , and λ is the parameter that controls the relative importance of constraints with respect to the primary cost term (i.e. function to be minimised). Initially, all penalty parameters of features are set to 0 ($p_{xi} = 0$, $p_{yj} = 0$, $i = 1, \dots, n$, $j = 1, \dots, m$). Each time local search settles in a local minimum, the penalty parameters of the features exhibited by the local minimum (only two at a time) are simply incremented by one. This increases by $2 \cdot \lambda$ the cost of all solutions that lie in the intersection of the zones corresponding to the penalised features and by λ the cost of all solutions that lie in either one of these zones (Fig 2). As a result, local search will primarily avoid the rectangular area, with centre the local minimum, and also to a lesser degree the two zones that run parallel to the co-ordinate axis as shown in Fig 2. This simple technique can be used to minimise arbitrary functions. In fact, there is nothing that binds the method to F6 which may not be used for other functions with two or more variables. The next section examines the results obtained for F6.

6. Experimentation with the F6 function

Following Davis [20], 22 bits were used for representing each variable. An equal number of features was used to cover the domain of each variable ($n = m$). The algorithm was relatively insensitive to the parameter λ and performed well for values of λ greater than 0.2. The value 0.25 for λ was used in the tests. Experiments were performed for varying n (i.e. number of features per variable) to determine how this parameter affects GLS. The values tried for n were 5, 10, 15, 20, 50, and 100. Fifty runs from random solutions

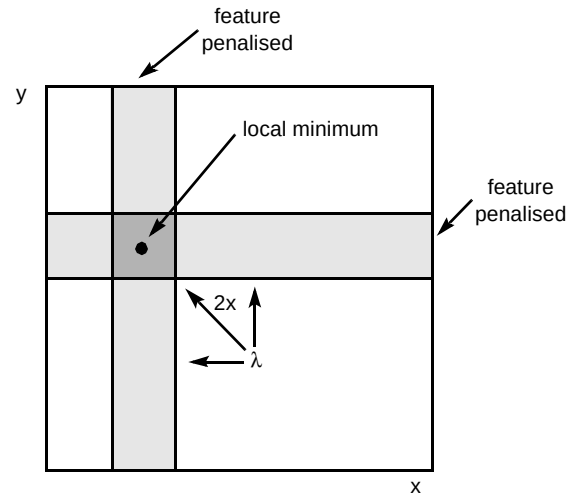


Fig 2 Changes in cost due to penalising the features exhibited by a local minimum.

(random binary strings) were performed for each value of n considered with the iteration limit set to 10 000 local search improvement cycles. Table 1 illustrates the results obtained. The best setting proved to be $n = m = 5$. Under this setting, the algorithm succeeded in finding the exact optimal solution (0,0) in 100% of 50 runs. Under all settings, the algorithms found the exact optimum many times.

This performance further improves if more time is given to the algorithm. For example, in the case ($n = m = 100$) where most failures occurred (28 out of 50 runs), the same experiment was performed but this time the algorithm was allowed to complete 100 000 local search iterations. The performance of GLS significantly improved and the algorithm found the exact optimum in 50 out of 50 runs (no failures).

The main observation made was that GLS performance degraded as the number of features used increased. More features meant more effort to leave a particular area but also more careful exploration. For this particular function, diversification of search to sample the whole search space proved important to find the global minimum quickly. The distribution of points visited for $n = m = 10$ during 10 000 iterations of local search is shown in Fig 3. During the particular run that generated Fig 3, the optimal solution was found early and after 1965 iterations. Despite that, the algorithm was allowed to continue until 10 000 iterations were completed to get a better picture of the solutions visited by the algorithm. As one can see from Figs 3 and 4 the algorithm distributed its efforts over the whole of the search space but visited mainly local minima. That is why points are arranged in concentric cycles around the point (0,0).

Table 1 GLS performance on F6 (time in CPU seconds on a DEC alpha 3000/600 175 MHz).

Number of features	$n = m = 5$	$n = m = 10$	$n = m = 15$	$n = m = 20$	$n = m = 50$	$n = m = 100$
Mean cost	0.00E+00	4.55E-11	3.19E-10	2.73E-10	1.97E-04	3.21E-04
Best solution	0.00E+00	0.00E+00	0.00E+00	0.00E+00	0.00E+00	0.00E+00
Worst solution	0.00E+00	2.28E-09	2.28E-09	2.28E-09	9.72E-03	9.72E-03
Mean iterations	2287.32	2566.22	2954.08	3526.9	4132.66	3738.48
Mean time	2.823333	3.150668	3.634334	4.382333	5.188333	4.654
Mean function evaluation	104958.6	117778.8	135588	161878.4	18975.6	171578.5
Optimal runs	50	49	43	44	31	22
Total runs	50	50	50	50	50	50

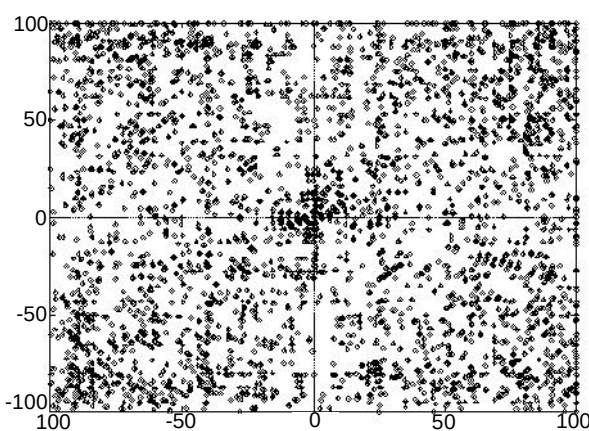


Fig 3 All the points visited during the first 10 000 iterations of local search.

This is more clearly demonstrated in Fig 4 where a 3-D view of the visited points is shown.

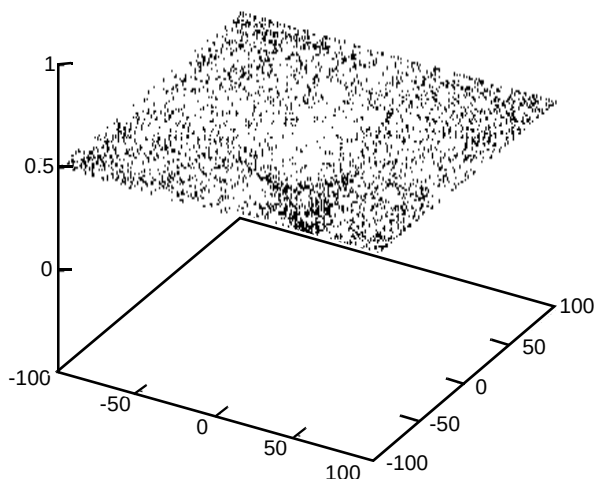


Fig 4 3-D view of Fig 3.

The shape formed is exactly the bottom part of F6 which suggests that the points are actually local minima in the great majority. Note here, that GLS is exploring binary space and not numeric space. In general, local minima and

their attraction basins in the binary space are different from the local minima and their attraction basins appearing in the numeric space. Because of the symmetrical landscape, the binary encoding used and the structure of the GLS features, the majority of the solutions visited by GLS in the case of F6 have the property of being numeric local minima as illustrated in Figs 3 and 4. This is not necessarily the case for functions with non-symmetrical landscapes. In these cases, grey encodings (see Battiti and Tecchioli [29] for example) and/or features of different structure may yield better performance than the encoding scheme and features used in this paper.

7. Conclusions

This paper has shown that GLS has the potential to be utilised in the optimisation of real-valued functions with numerous local minima, which are considered to be difficult for gradient-based methods. The application of GLS to optimise the F6 function, a benchmark for Genetic Algorithms, has been examined. GLS repeatedly located the exact global optimum of the function. The paper also serves in demonstrating how artificial solution features can be created when no features can be deduced from the structure of the objective function, which adds support to the claim that GLS has wide applications.

References

- 1 Voudouris C and Tsang E: 'Partial constraint satisfaction problems and guided local search', Proceedings of 2nd International Conference on Practical Application of Constraint Technology (PACT'96), London, pp 337—356 (April 1996).
- 2 Tsang E and Voudouris C: 'Fast local search and guided local search and their application to BT's workforce scheduling problem', Operations Research Letters, 20, No 3, pp 119—127 (1997).
- 3 Backer B D, Furnon V, Prosser P, Kilby P and Shaw P: 'Solving vehicle routing problems using constraint programming and metaheuristics', Submitted to the Journal of Heuristics, special issue on Constraint Programming.

- 4 Kilby P, Prosser P and Shaw P: 'Guided local search for the vehicle routing problem', *Proceedings of the 2nd International Conference of Metaheuristics* (July 1997).
- 5 Voudouris C: 'Guided local search for combinatorial optimization problems', PhD Thesis, Department of Computer Science, University of Essex, Colchester, UK (July 1997).
- 6 Osman I H: 'An introduction to meta-heuristics', in Lawrence M and Wilson C (Eds): 'Operational research tutorial papers', Operational Research Society Press, Birmingham, UK, pp 92—122 (1995).
- 7 Reeves C R (Ed): 'Modern Heuristic Techniques for Combinatorial Problems', Blackwell Scientific Publishing (1993).
- 8 Reeves C R: 'Modern heuristic techniques', in Rayward-Smith V J, Osman I H, Reeves C R and Smith G D (Eds): 'Modern Heuristic Search Methods', John Wiley & Sons, pp 1—25 (1996).
- 9 Glover F: 'Future paths for integer programming and links to artificial intelligence', *Computers Ops Res*, 5, pp 533—549 (1986).
- 10 Glover F: 'Tabu search and adaptive memory programming — advances, applications and challenges', in Barr, Helgason and Kennington (Eds): 'Interfaces in Computer Science and Operations Research', Kluwer Academic Publishers (1996).
- 11 Glover F: 'Tabu search fundamentals and uses', Graduate School of Business, University of Colorado, Boulder (1995).
- 12 Glover F: 'Tabu search Part I', *ORSA Journal on Computing*, 1, pp 190—206 (1989).
- 13 Glover F: 'Tabu search Part II', *ORSA Journal on Computing*, 2, pp 4—32 (1990).
- 14 Glover F: 'Tabu search: improved solution alternatives for real world problems', in Birge and Murty (Ed): 'Mathematical Programming: State-of-the-art', pp 64—92 (1994).
- 15 Aarts E H L and Korst J H M: 'Simulated Annealing and Boltzmann Machines', Wiley, Chichester (1989).
- 16 Dowsland A: 'Simulated annealing', in Reeves C R (Ed): 'Modern Heuristic Techniques for Combinatorial Problems', Blackwell Scientific Publishing, pp 20—69 (1993).
- 17 Kirkpatrick S, Gelatt C D and Vecchi M P: 'Optimisation by simulated annealing', *Science*, 220, pp 671—680 (1983).
- 18 Laarhoven P J M V and Aarts E H L: 'Simulated Annealing: Theory and Application', Kluwer, Dordrecht (1988).
- 19 Feo T A and Resende M G C: 'Greedy randomized adaptive search procedures', *Journal of Global Optimization*, 6, pp 109—133 (1995).
- 20 Davis L: 'Handbook of genetic algorithms', Van Nostrand Reinhold (1991).
- 21 Goldberg D E: 'Genetic algorithms in search, optimisation, and machine learning', Addison-Wesley (1989).
- 22 Reeves C R: 'Genetic Algorithms', in Reeves C (Ed): 'Modern Heuristic Techniques for Combinatorial Problems', Blackwell Scientific Publishing, pp 151—196 (1993).
- 23 Glover F and Laguna M: 'Tabu search', in Reeves C R (Ed): 'Modern Heuristic Techniques for Combinatorial Problems', Blackwell Scientific Publishing, pp 71—141 (1993).
- 24 Hiller F S and Lieberman G: 'Introduction to operations research', Sixth edition, McGraw-Hill, New York (1995).
- 25 Holland J H: 'Adaptation in natural and artificial systems', University of Michigan Press, Ann Arbor, MI (1975).
- 26 Ingber A L: 'Very fast simulated re-annealing', *Journal of Mathematical Computer Modelling*, 12, No 8, pp 967—973 (1989).
- 27 Styblinski M A and Tang T S: 'Experiments in non-convex optimisation: stochastic approximation with function smoothing and simulated annealing', *Neural Networks*, 3, pp 467—483 (1990).
- 28 Kota S and Chiou S: 'Use of orthogonal arrays in mechanism synthesis', *Mechanical Machine Theory*, 28, pp 777—794 (1993).
- 29 Battiti R and Tecchioli G: 'The reactive tabu search', *ORSA Journal on Computing*, 6, pp 126—140 (1994).
- 30 Woodruff D L and Zemel E: 'Hashing vectors for tabu search', *Annals of Operations Research*, 41, pp 123—137 (1993).
- 31 Schaffer J D, Caruana R A, Eshelman L J and Das R: 'A study of control parameters affecting online performance of genetic algorithms for function optimisation', *Proceedings of 3rd International Conference on Genetic Algorithms*, Morgan Kaufmann, pp 51—60 (1989).



Christos Voudouris holds a Diploma degree in Electrical and Computer Engineering from the National Technical University of Athens (1992), an MSc in Intelligent Knowledge Based Systems from the University of Essex (1993), and a PhD in Computer Science from the University of Essex (1997). During the years 1994—1995, he worked as a senior research officer in the Department of Computer Science at the University of Essex conducting research on heuristic search techniques for combinatorial optimisation problems. He joined the Intelligent Systems Research group at BT Laboratories in October 1995. Since then he has been working on research projects in the areas of resource scheduling, intelligent agents, and machine intelligence. He currently leads a project on the use of constraint optimisation techniques to resource scheduling applications. His research interests are combinatorial optimisation, heuristic search techniques, dynamic scheduling, intelligent agents, and fuzzy logic.